Tutorial Per File — Flutter CRUD dengan JSONPlaceholder

Dokumen ini menjelaskan bagaimana project Hari 2 dibuat dari bawah ke atas. Urutannya sengaja mengikuti dependency: model lebih dahulu, kemudian adapter, HTTP client, repository, controller, UI, dan test.

Arsitektur akhir

```
Home/Form Screen
      ↓ intent + render state
InspectionController
      ↓ use case/state mutation
InspectionRepository
      ↓ remote/cache policy
ApiClient
      ↓ facade ke 4 operasi terpisah
Read / Create / Update / Delete
      ↓ HTTP + status + JSON
JSONPlaceholder /posts

JsonPlaceholderMapper berada antara JSON publik dan model domain
Inspection.
```

1. `pubspec.yaml` — dependency

Tambahkan package:

```
fvm flutter pub add http shared_preferences image_picker file_picker
```

- `http`: request GET/POST/PUT/DELETE.
- `shared_preferences`: cache daftar terakhir.
- `image_picker`: memilih image dari galeri.
- `file_picker`: memilih dokumen generik.

Simpan `pubspec.lock` agar seluruh peserta memakai dependency yang sama.

2. `lib/src/models/inspection.dart` — bahasa domain

Model aplikasi tetap memakai istilah inspeksi dan tidak dipaksa menjadi `Post`.

```dart
class Inspection {
  const Inspection({
    required this.id,
    required this.assetCode,
    required this.location,
    required this.condition,
    required this.note,
```

```
    required this.createdAt,
  });

  final String id;
  final String assetCode;
  final String location;
  final InspectionCondition condition;
  final String note;
  final DateTime createdAt;
}
```

`fromJson`/`toJson` pada model dipakai untuk cache internal. JSONPlaceholder tidak dibaca langsung di sini karena schema eksternal adalah concern adapter.

Checkpoint: buat unit test round-trip model sebelum membuat request.

3. `lib/src/data/json_placeholder_mapper.dart` — schema adapter

JSONPlaceholder `/posts` mengirim:

```
{
  "userId": 1,
  "id": 1,
  "title": "...",
  "body": "..."
}
```

Sedangkan aplikasi membutuhkan kode aset, lokasi, kondisi, dan catatan. Mapper menjembatani keduanya:

```
static Inspection fromPost(Map<String, dynamic> json) {
  final id = json['id'].toString();
  final number = int.tryParse(id) ?? 0;
  return Inspection(
    id: id,
    assetCode: json['assetCode'] ?? 'POST-$
{number.toString().padLeft(3, '0')}',
    location: json['location'] ?? 'User ${json['userId']}',
    condition: number.isEven
        ? InspectionCondition.good
        : InspectionCondition.needsAction,
    note: (json['note'] ?? json['body'] ?? '').toString(),
    createdAt: DateTime.utc(2026).add(Duration(days: number)),
  );
}
```

Untuk write, `toPost` mengirim field standar JSONPlaceholder (`title`, `body`, `userId`) sekaligus field domain. Layanan mengembalikan field tambahan itu, sehingga simulated response dapat dipetakan kembali ke `Inspection`.

Mengapa file terpisah? Bila API diganti, model dan UI tidak perlu ikut mengetahui nama `title/body/userId`.

4. `lib/src/data/crud/` — empat operasi, empat file

CRUD sengaja dipisah agar satu operasi dapat dijelaskan, dipanggil, dan diuji tanpa membuka satu file besar:

```
data/crud/
├── read_inspections.dart
├── create_inspection.dart
├── update_inspection.dart
└── delete_inspection.dart
```

Setiap class bersifat callable. Contoh Read:

```dart
class ReadInspections {
  const ReadInspections({required this.client, required
this.baseUrl});

  final http.Client client;
  final String baseUrl;

  Future<List<Inspection>> call() async {
    final response = await client.get(
      Uri.parse('$baseUrl/posts?_limit=20'),
    );
    final posts = JsonResponseDecoder.list(response, expectedStatus:
200);
    return posts.map(JsonPlaceholderMapper.fromPost).toList();
  }
}
```

Urutan bedah yang disarankan:

1. Read: URL, query, status `200`, list decode.
2. Create: header JSON, mapper payload, status `201`.
3. Update: stable ID pada path/payload, status `200`.
4. Delete: konfirmasi di UI, status `200/204`, tanpa decode model.

`api_exception.dart` dan `json_response_decoder.dart` adalah helper bersama agar empat file tidak menduplikasi parsing error.

5. `lib/src/data/api_client.dart` — facade yang mudah dipanggil

Base URL default:

```dart
const apiBaseUrl = String.fromEnvironment(
  'API_BASE_URL',
  defaultValue: 'https://jsonplaceholder.typicode.com',
);
```

```dart
Future<List<Inspection>> fetchInspections() => read();
Future<Inspection> createInspection(fields) => create(fields);
Future<Inspection> updateInspection(id, fields) => update(id, fields);
Future<void> deleteInspection(id) => delete(id);
```

Repository tetap memanggil method facade yang jelas. Saat demonstrasi, trainer juga dapat memanggil operation object seperti `api.read()` atau `api.create()`. File ini tidak mengatur widget atau memodifikasi list UI.

Checkpoint: uji dengan `MockClient`; jangan bergantung pada internet dalam automated test.

6. `lib/src/data/inspection_repository.dart` — policy data

Repository menyembunyikan sumber data dari controller:

```dart
Future<InspectionResult> list() async {
  try {
    final items = await api.fetchInspections();
    await writeCache(items);
    return remoteResult;
  } on TimeoutException {
    return await readCache() ?? throw timeoutFailure;
  }
}
```

Write diteruskan ke API. Cache hanya dipakai untuk GET ketika jaringan gagal. Cached mode dibuat read-only agar peserta tidak mengira write offline akan tersinkron otomatis.

7. `lib/src/controllers/inspection_controller.dart` — state dan intent

Controller mempunyai state eksplisit:

```
items
isLoading
isSubmitting
isCached
fetchedAt
errorMessage
```

- `load()` mengganti list dari API/cache.
- `create()` memasukkan simulated response ke list lokal.
- `update()` mengganti item dengan ID yang sama.
- `delete()` menghapus item lokal setelah response sukses.

Perubahan write akan hilang saat `load()` berikutnya karena JSONPlaceholder tidak menyimpan write. Ini perilaku yang sengaja dipakai sebagai bahan diskusi.

8. `lib/src/app.dart` — composition root

`app.dart` membuat dependency satu kali:

```
_api = ApiClient(baseUrl: apiBaseUrl);
_repository = InspectionRepository(_api);

home: HomeScreen(repository: _repository)
```

Tidak ada login karena public endpoint tidak memerlukan authentication. `ApiClient.close()` tetap dipanggil saat aplikasi dispose.

9. `lib/src/screens/home_screen.dart` — render state

Home screen:

- memanggil `controller.load()` dari `initState`, bukan dari `build`;
- menampilkan loading, data, empty, error, cache, dan refresh;
- menyediakan menu Update/Delete dan FAB Create;
- menampilkan banner bahwa write hanya simulasi;
- menyediakan tombol reset yang memuat ulang data public API.

UI tidak membaca status HTTP atau melakukan `jsonDecode`; semua itu milik `ApiClient`.

10. `lib/src/screens/inspection_form_screen.dart` — form dan picker

Form dipakai untuk Create dan Update. `initialItem` menentukan mode edit.

```
final result = initialItem == null
    ? await controller.create(fields)
    : await controller.update(initialItem!.id, fields);
```

Image/file picker hanya mengubah state lokal. File tidak dimasukkan ke body JSONPlaceholder dan UI menampilkan label “belum di-upload”.

Contoh `image_picker` untuk galeri:

```
final selected = await ImagePicker().pickImage(
  source: ImageSource.gallery,
  maxWidth: 1600,
  imageQuality: 82,
);
if (!mounted || selected == null) return;
setState(() {
  _image = selected;
  _file = null;
});
```

Contoh `file_picker` untuk dokumen umum:

```
final selected = await FilePicker.pickFile();
if (!mounted || selected == null) return;
setState(() {
  _file = selected;
```

```
  _image = null;
});
```

Kedua contoh menangani cancel sebagai kondisi normal. `XFile` cocok untuk preview gambar, sedangkan `PlatformFile` menyediakan nama dan metadata file. Pemilihan file bukan upload; multipart upload sengaja berada di luar scope.

11. `test/json_placeholder_api_test.dart` — contract adapter test

Test memakai `MockClient` untuk membuktikan:

- GET memakai `/posts?_limit=20` dan mapping post → inspection benar;
- POST mengirim `title/body/userId` serta field domain;
- PUT membawa ID yang sama pada path/payload;
- DELETE menerima status 200 dari JSONPlaceholder.

Test tidak menghubungi internet sehingga cepat dan deterministik.

12. Verifikasi endpoint public secara manual

```
curl 'https://jsonplaceholder.typicode.com/posts?_limit=2'

curl -X POST 'https://jsonplaceholder.typicode.com/posts' \
  -H 'Content-Type: application/json' \
  -d '{"title":"PMP-900","body":"Simulasi","userId":1}'

curl -X PUT 'https://jsonplaceholder.typicode.com/posts/1' \
  -H 'Content-Type: application/json' \
  -d '{"id":1,"title":"PMP-001","body":"Update","userId":1}'

curl -X DELETE 'https://jsonplaceholder.typicode.com/posts/1'
```

Status yang diverifikasi pada saat materi dibuat: GET 200, POST 201, PUT 200, dan DELETE 200.

13. Quality gate

```
fvm dart format --output=none --set-exit-if-changed lib test
fvm flutter analyze
fvm flutter test
fvm flutter build apk --debug
```

Challenge peserta

1. Ubah `_limit=20` menjadi parameter repository yang tervalidasi.
2. Tambahkan pencarian lokal berdasarkan kode/lokasi.
3. Tambahkan badge `SIMULATED` pada hasil create/update.
4. Buat fake response 500 dan pastikan cache/retry tampil benar.
5. Ganti mapper agar resource `/todos` menjadi work order tanpa mengubah UI.
6. Tulis PRD perubahan dari simulasi ke backend persistence nyata.